

開発説明書

LiQt Project

```
    // luDec(DcmpType epsilon = 1e-9); ///< LU dec  
    (DcmpType epsilon = 1e-9); ///< Inverse matrix  
    epsilon = 1e-9); ///< Determinant
```

```
    typename CopyType1, typename CopyType2>  
    Matr ix_(Matr ixType<CopyType1>&, const Matr ixType<C
```

```
    int ixType<typename Type_ = Type>
```



フロントエンド
バックエンド

フロントエンド

- フロントエンドとは
ブラウザ側(表示部分)で動くプログラムのこと

- HTML

```
<html>
  <body>
    <h1>HELLO!!</h1>
  </body>
</html>
```

- CSS

```
body{
  background-color:red;
  color:white;
}
```

- JavaScript

```
const h1
=document.getElementsByTagName('h1')[0];
h1.style.color="black"
```

データの表示部分だけ

バックエンド

- バックエンドとは
サーバ側で動くプログラムのこと

- PHP

```
<?php  
echo 'hello';  
?>
```

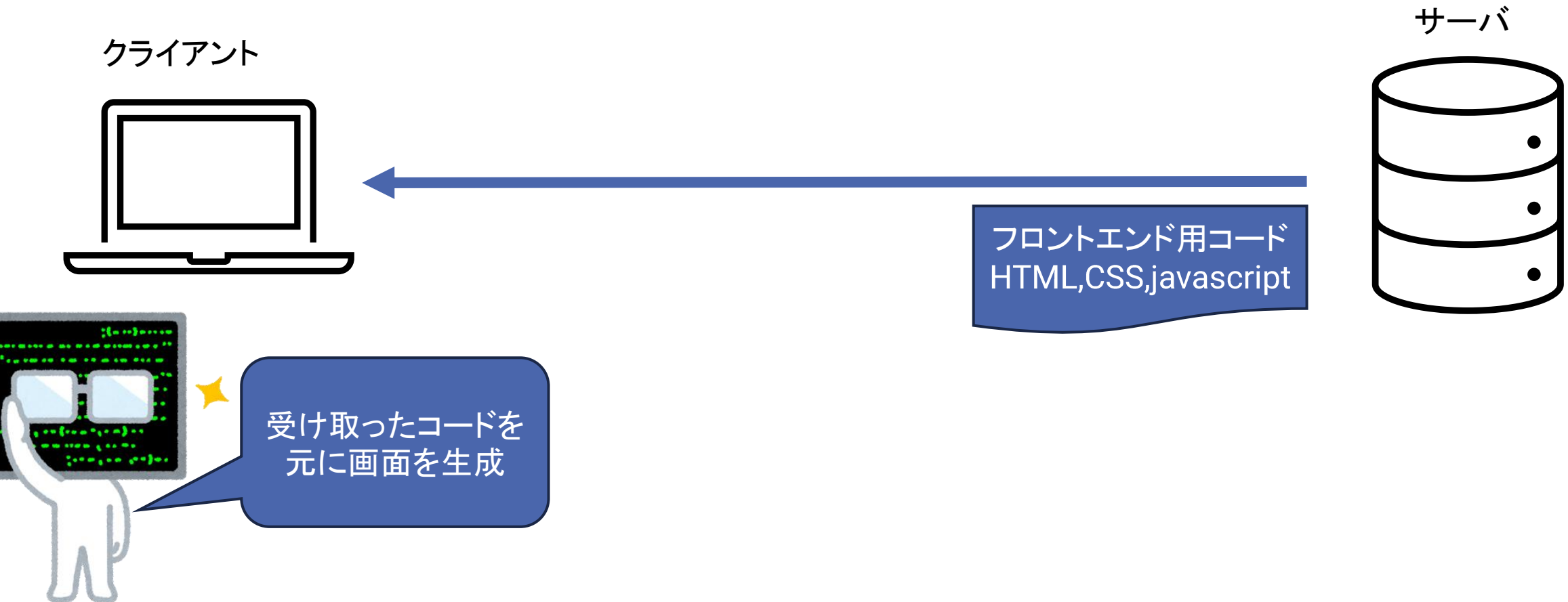
- SQL

```
SELECT * FROM users;  
INSERT INTO users(name)  
VALUES('SHIMADA')
```

認証やデータの保存など

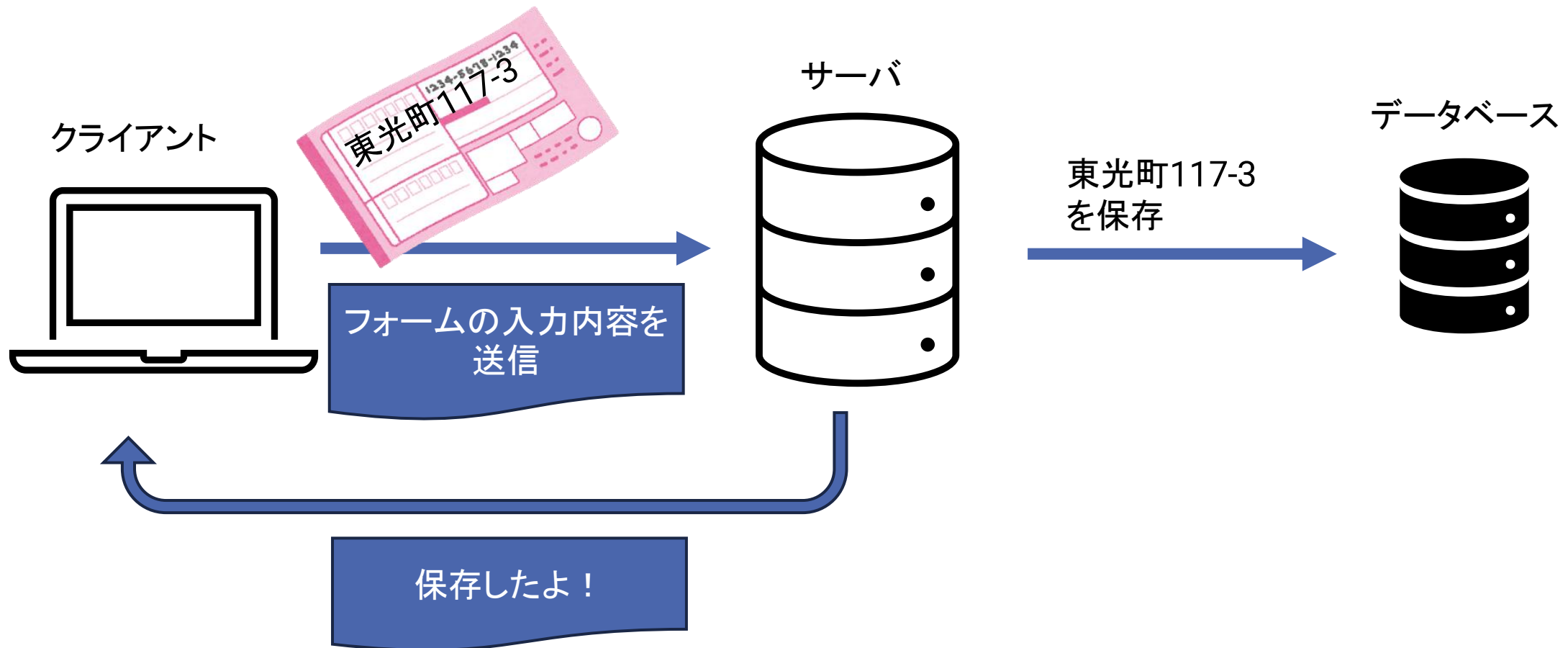
フロントエンドとバックエンド

- フロントエンドとバックエンドの流れ
フロントエンド



フロントエンドとバックエンド

- フロントエンドとバックエンドの流れ

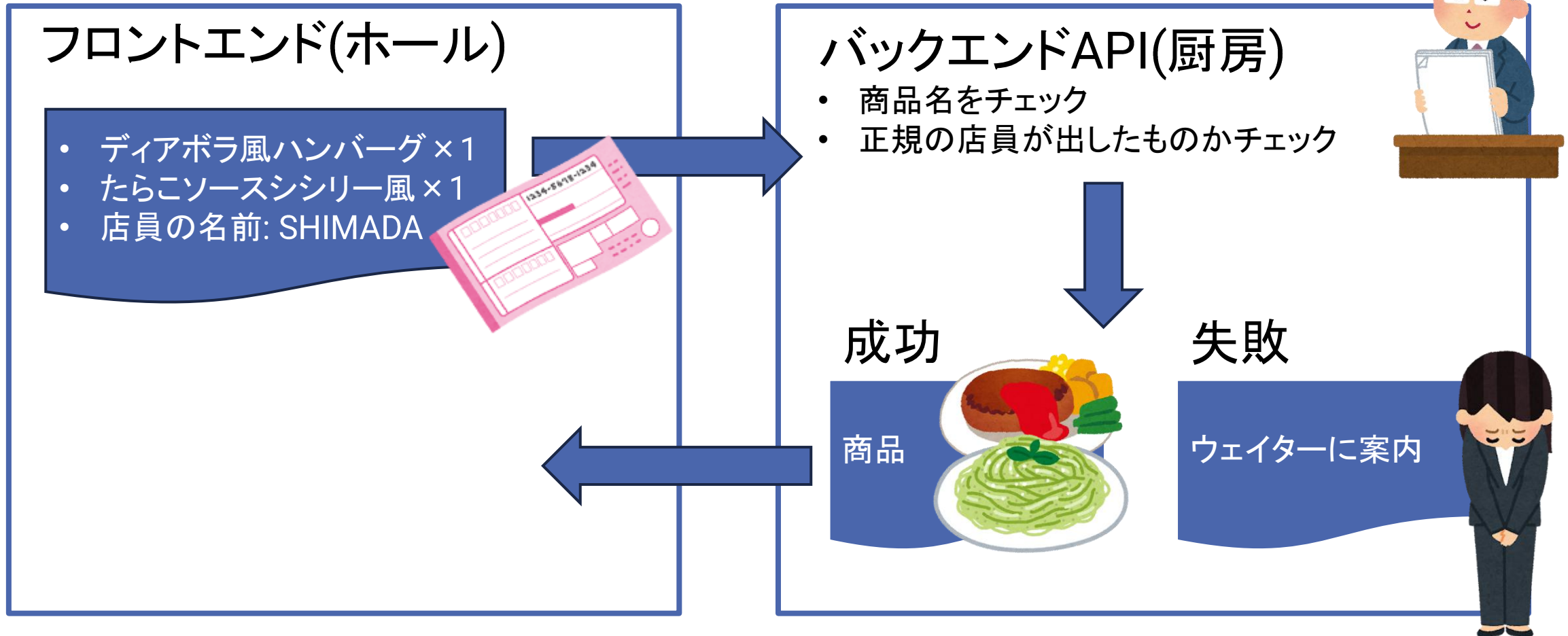


The background features a dark blue area on the left and a light blue area on the right, separated by a diagonal line. A network diagram with circular nodes and connecting lines is overlaid on the dark blue section.

開発手法

API方式

- API=レストラン



API方式

- フロントエンドからバックエンドAPI(PHP)に対しリクエスト

フロントエンド(login.html)

```
{  
  "mail"      : "simada",  
  "password"  : "simadaaa",  
  "csrf_token": "token..."  
}
```

成功 !
or
失敗 !

バックエンドAPI(api/login.php)

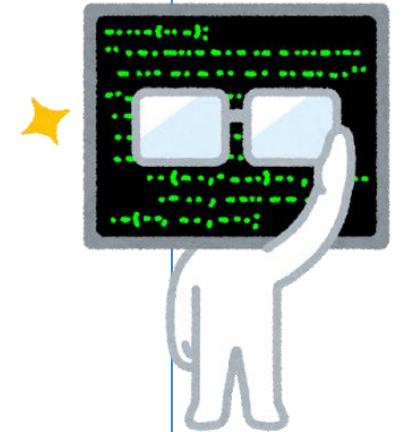
- Mailアドレスとパスワードが合っているか検証
- 成功の場合Sessionに追加
- CSRFトークンを検証

成功

```
{  
  "success": true  
}
```

失敗

```
{  
  "success": false  
}
```

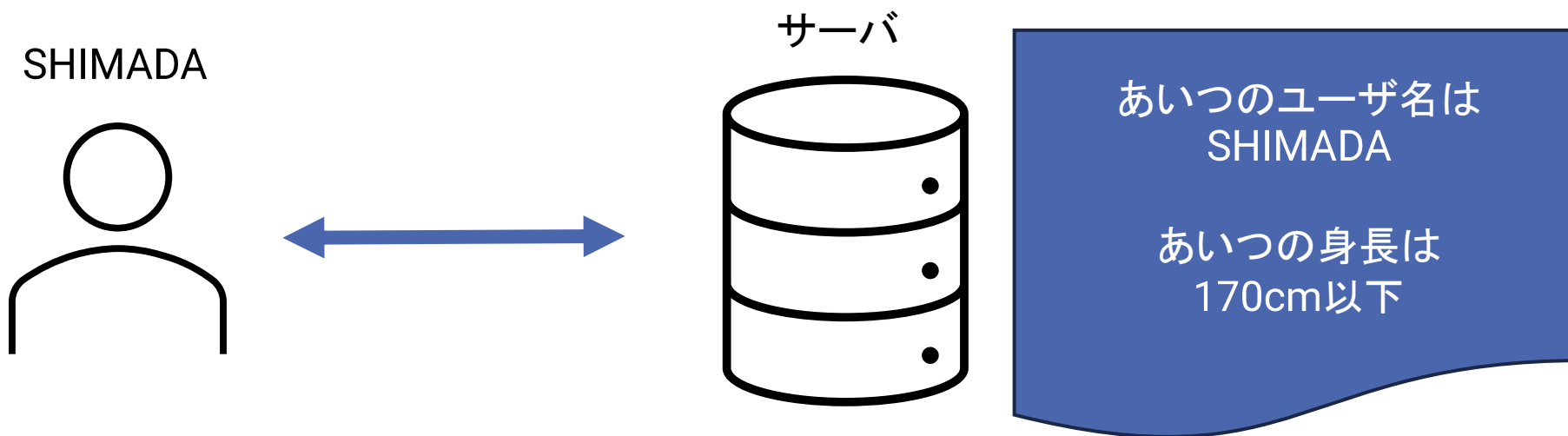




PHPの復習 Session

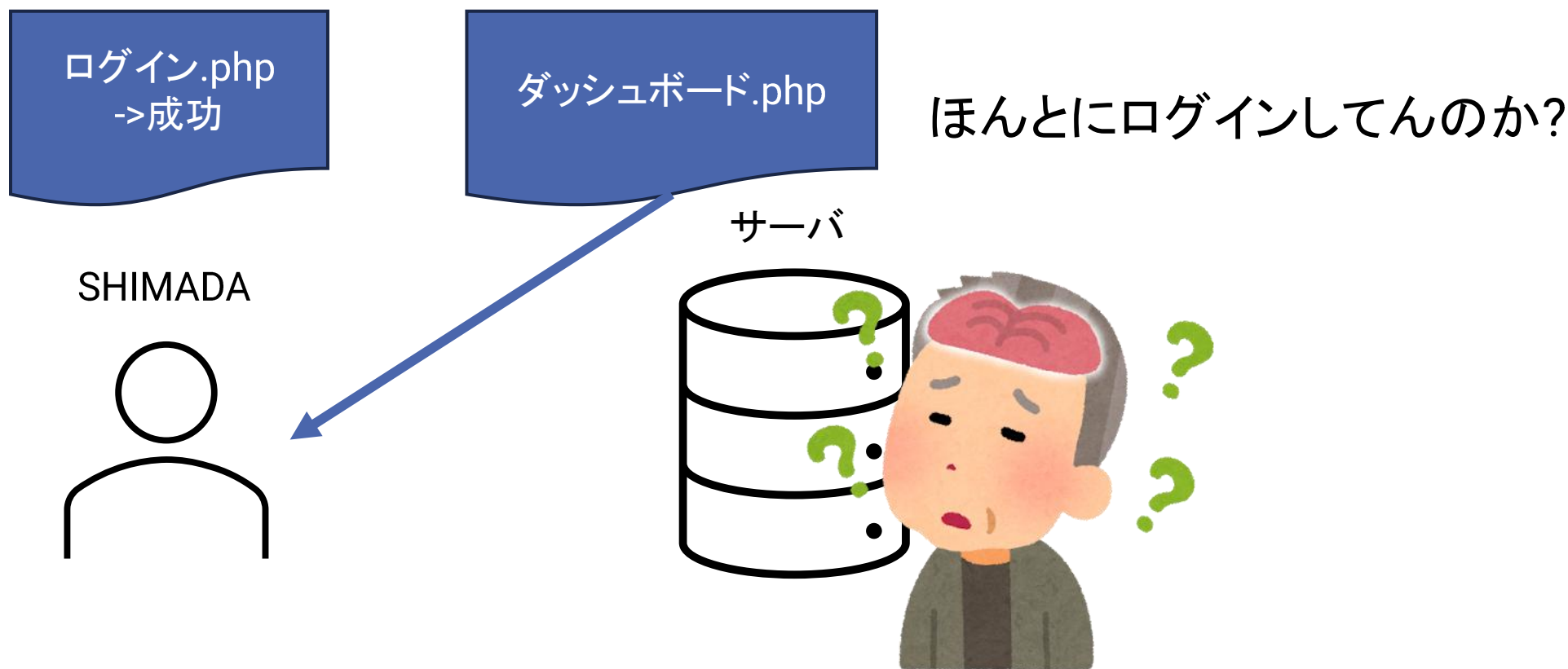
PHPの復習 Session

- Sessionとは
クライアントの情報をサーバが保持しておく領域



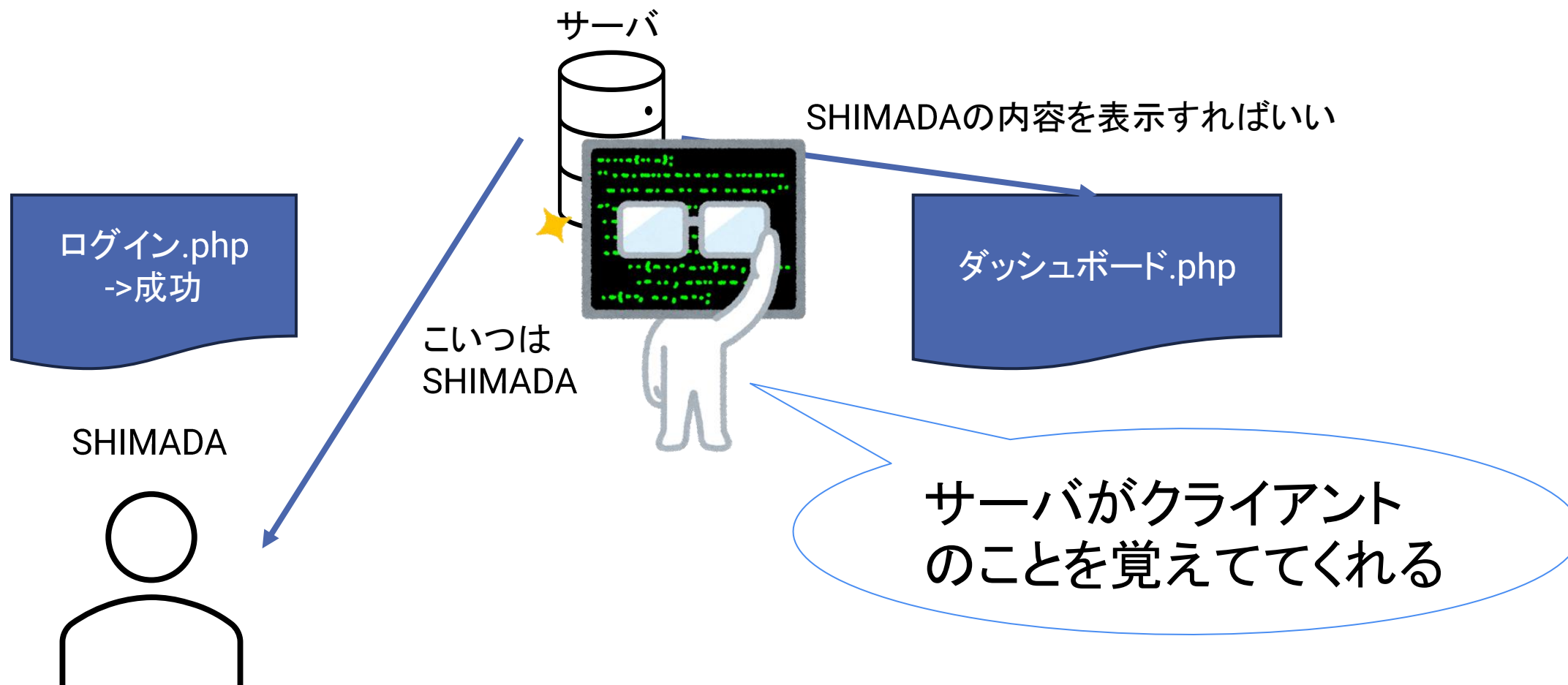
なぜSessionを使うのか

- ログインが一度で済む
セッションに保存しない場合
ページごとにアカウント名とパスワードを認証する必要がある



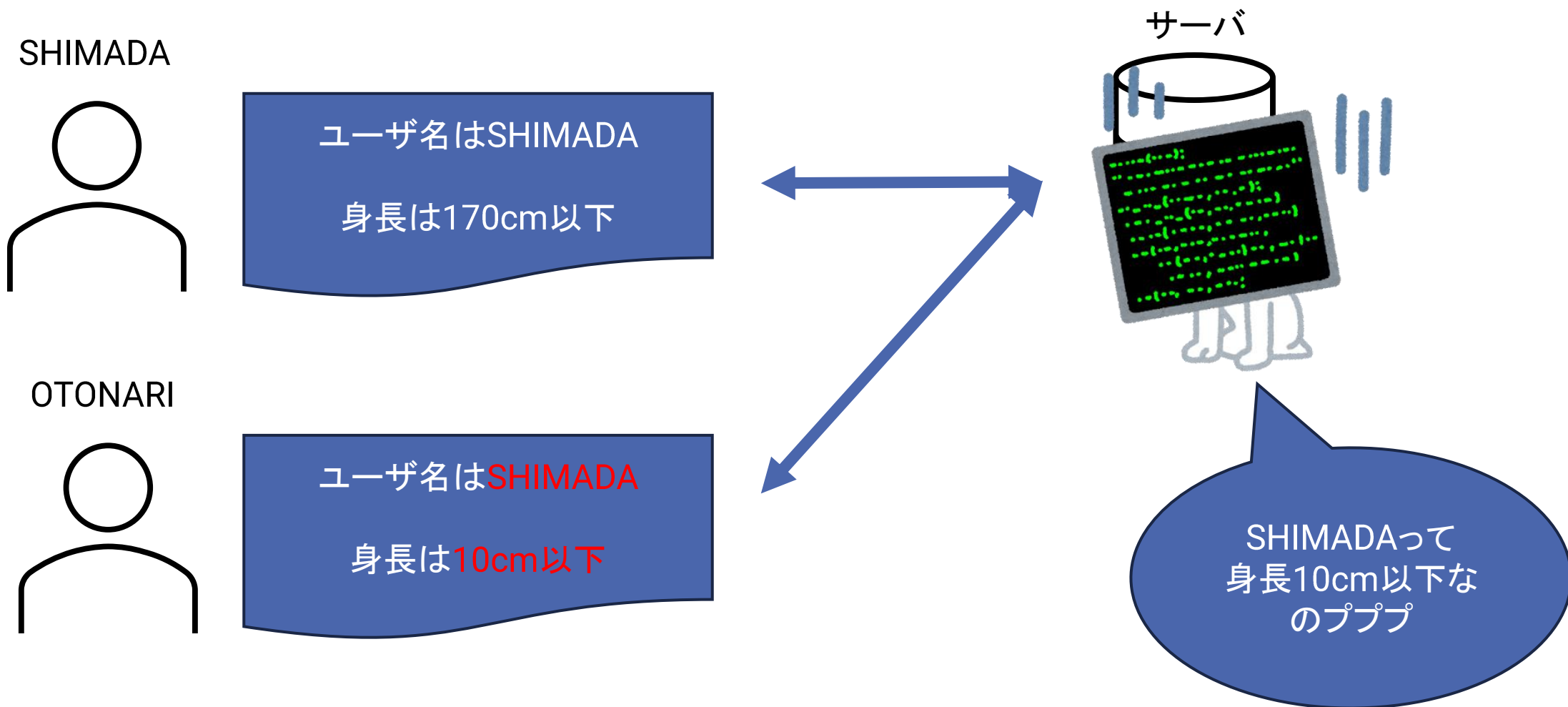
なぜSessionを使うのか

- ログインが一度で済む



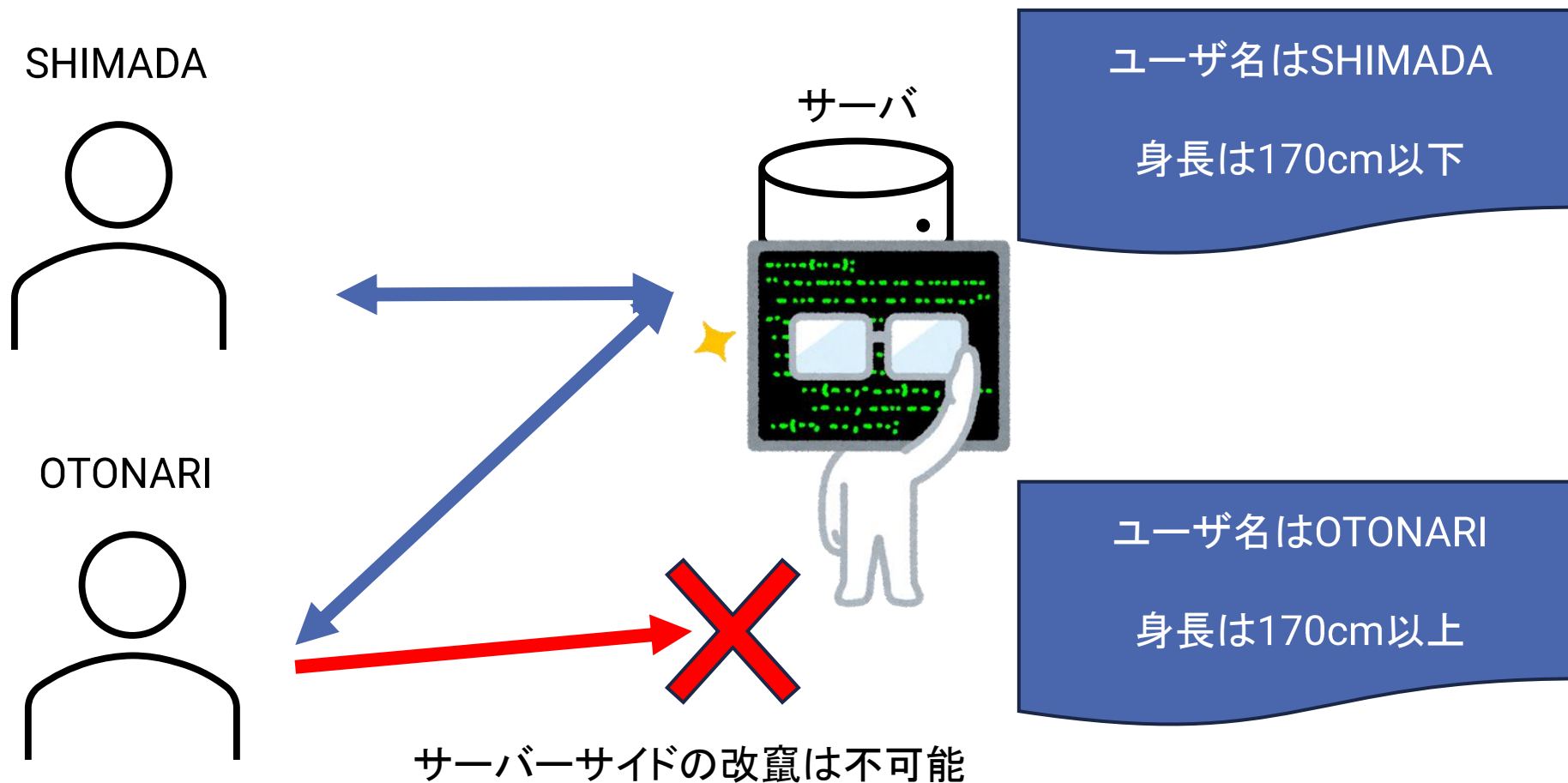
なぜSessionを使うのか

- クライアントで保持していると改ざんし放題



なぜSessionを使うのか

- クライアントで保持していると改ざんし放題

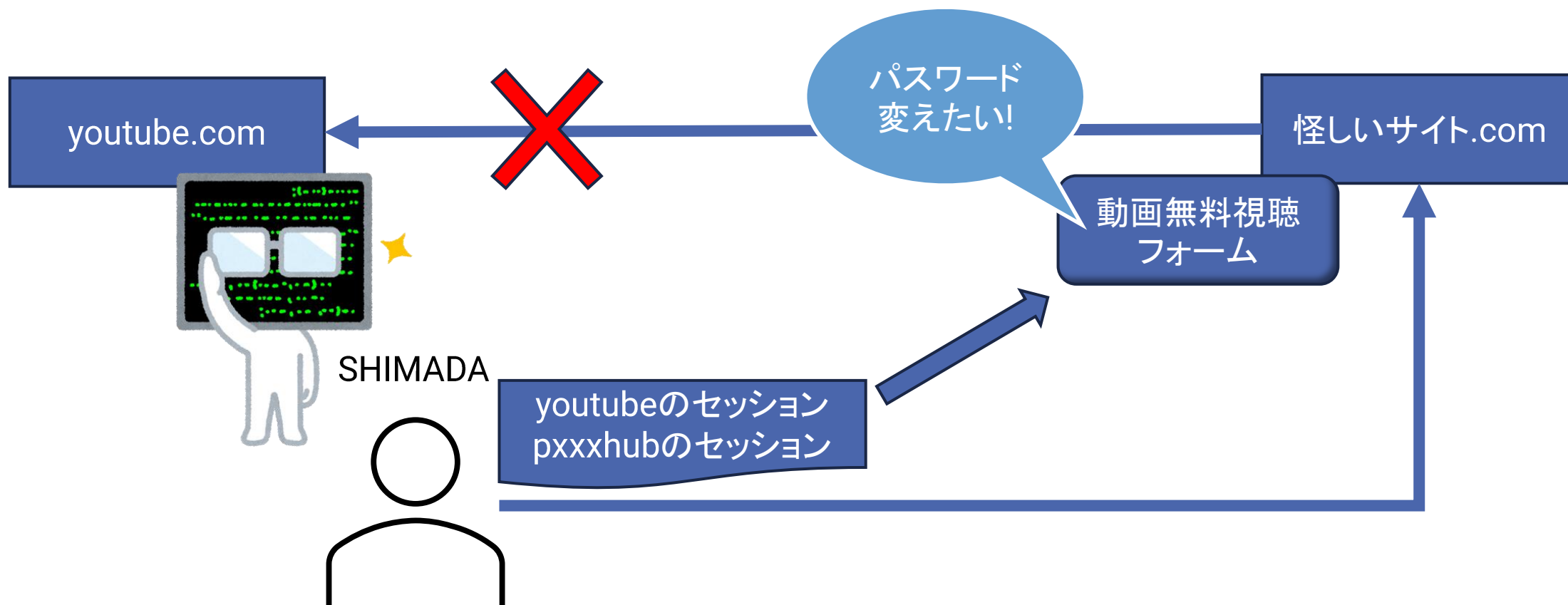




PHPの復習 CSRFトークン

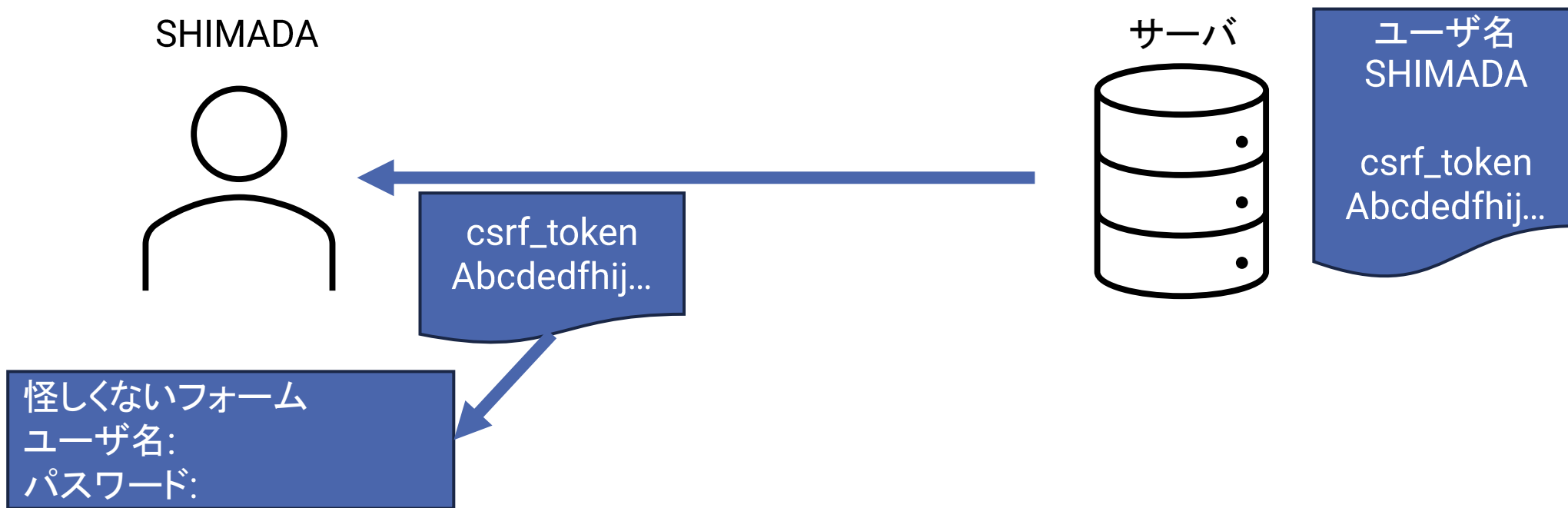
CSRFトークンとは

- CSRF(Cross Site Request Forgeries)トークン
正規のフォームから送られているかを検証する



CSRFトークン使い方

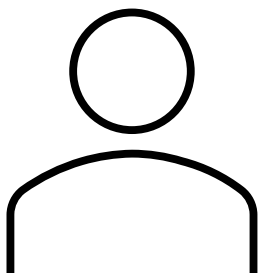
- ・フォームにバックエンドで生成されたトークンをフォームに埋め込む



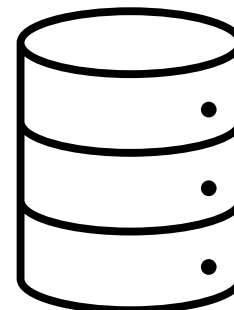
CSRFトークン使い方

- 検証

SHIMADA



サーバ



ユーザ名
SHIMADA

csrf_token
Abcdedfhij...

怪しくないフォーム
ユーザ名:
パスワード:
csrf_token: abcdefghij...

csrfトークンが一致したということは
正規のフォームから入れられたもの

使用ライブラリの説明

- ライブラリを使用するためにComposerのautoload.phpを読み込んでください。

```
require_once __DIR__ . '/path/to/autoload.php';
```

＊環境はすべて整えてあります。

- lib¥Session: セッションを自動で管理する独自ライブラリ

```
$s = new Session();           // Sessionのインスタンス生成
```

*リメンバートークンを自動で検証します

```
$s->getCurrentUserId();       // ログイン中のIDを取得(数値/null)
```

```
$s->isLoggedIn();             // 現在ログイン中か判定(true/false)
```

```
$s->tryLogin('メールアドレス','パスワード',リメンバートークンを使用するか);  
// ログインを試みる(true/false)
```

*リメンバートークン: セッションが切れてもログイン状態を維持する。

*ログインに成功した場合true/失敗した場合falseを返します。

- lib¥CSRFToken: トークンを生成/検証する独自ライブラリ

```
$csrfToken = new CSRFToken();
```

```
$csrfToken->getToken(); // トークンを取得=フォームに埋め込む
```

```
$csrfToken->isValid(トークン); // トークンを検証=バックエンドで検証
```

• フロントエンド

```
<?php $csrfToken = new lib¥CSRFToken(); ?>
<form>
  <input
    type="hidden"
    value="<?= $csrfToken->getToken() ?>"
    name="csrf_token"
  />
  <input type="mail" name="mail" />
  <input type="submit" />
</form>
```

• バックエンド

```
<?php
$csrfToken = new lib¥CSRFToken();
$csrf_token = $_POST['csrf_token'];
$mail       = $_POST['mail'];

if($csrfToken->isValid($csrf_token))
  echo '問題なし';
else
  echo '不正なリクエスト';
?>
```

- Phinx: データベースのマイグレーションを行うライブラリ

データベースのスキーマをバージョン管理するためのツールで、**データベースの変更をコードで管理**できます。

これにより、開発チーム全体でデータベースの変更を共有しやすくなります。

マイグレーションファイルは、データベースの変更を定義するPHPクラスで構成されます。

seedsによって**初期データ、テストデータを投入することもできます。**

最新のデータベースにする。 : vendor/bin/phinx migrate

前に戻す。 : vendor/bin/phinx rollback

マイグレーションを作成する。 : vendor/bin/phinx create **マイグレーション名**

Seedを作成する。 : vendor/bin/phinx seed:create **シード名**

Seedを適用する。 : vendor/bin/phinx seed:run

- データベースをモデルとして扱うlaravelのライブラリ

Modelを継承してモデルを作成する。=Laravelを使っていれば自動で作ってくれる。

- SQL

```
SELECT id, user_id, password FROM users;
```

```
SELECT id, user_id FROM users  
WHERE user_id = 'otonari';
```

```
UPDATE users SET user_id = 'otonari'  
WHERE id = 1;
```

```
INSERT INTO users (id, user_id) VALUES ( 1,  
'otonari');
```

- ORM

- プレースホルダを自動で入れてくれる。
- 直感的な操作ができる。
- DBに依存しないコードを書ける。

```
models¥User::all(['id', 'user_id', 'password']);
```

```
models¥User::query()  
->where('user_id', 'otonari')  
->get(['id', 'user_id']); // 1件だけの場合はfirst
```

```
models¥User::query()  
->where('id', 1)  
->update(['user_id' => 'otonari']);
```

```
models¥User::create([  
    'id' => 1, 'user_id' => 'otonari',  
]);
```